Laboratorio de Programación (LPR) — 5° Año 3° Grupos A-B

EEST N.º 1 – "Eduardo Ader" — Vicente López

ACTIVIDAD 7: MINI JUEGO "PIEDRA, PAPEL O TIJERA" CON PERSISTENCIA DE DATOS (struct + fstream)

Informe individual — 2° Cuatrimestre 2026 Normas APA 7.ª ed.

Modalidad: Individual

Versión: v1.0.0 — Fecha de entrega: 16 de Octubre de 2026

Link del repositorio: https://github.com/teolaguna/piedrapapelotijera.git

Estudiante (Nombre y Apellido)	**Teo Laguna**
Curso / Grupo	5° Año 3° Grupo: A / B
Materia / Horario	Laboratorio de Programación — Miércoles y jueves de 15:15 a 17:15 h
Profesor	Mansilla Muñoz York Elías y Yamil Ganduglia
Repositorio	piedrapapelotijera/ (src/, docs/, docs/manuales/, capturas/)
Fecha de entrega	16/10/2026

Resumen

Se desarrolló de forma individual el Mini Juego de Piedra, Papel o Tijera contra la IA de la consola, con registro automático de cada partida en una estructura struct Partida y persistencia en disco en el archivo src/historial_partidas.txt mediante la librería <fstream>. La escritura usa ofstream en modo ios::app (agregado sin borrado) y la lectura usa ifstream con parseo CSV. El programa compila con g++ -std=c++11 y fue verificado en Windows 10/11 con PowerShell en VS Code. Este informe documenta el desarrollo, responde el Cuestionario Técnico de Control y reflexiona sobre la aplicación de la técnica al Proyecto Integrador Anual desde una perspectiva individual.

Palabras clave: C++, struct, fstream, ofstream, ifstream, ios::app, persistencia, Piedra Papel o Tijera.

1. Introducción y objetivos

Muchos programadores junior confunden la memoria RAM (volátil: los datos se pierden al cerrar el programa) con el almacenamiento secundario o disco (no volátil: los archivos permanecen). Esta actividad demuestra la diferencia guardando cada partida en disco y recuperándola aun después de cerrar la consola.

1.1 Objetivos clave

1. Modelado con struct: agrupar ID, Jugador, EleccionJugador, EleccionIA y Resultado en una sola entidad Partida.
2. Escritura en disco (ofstream): persistir en modo append (ios::app) sin sobrescribir partidas previas.
3. Lectura desde disco (ifstream): recuperar el historial completo y mostrarlo en pantalla (Opción 2).

1.2 Alcance individual

Trabajo individual: se desarrolla el juego tal cual lo provisto en la Actividad 7, sin adaptación al P.I.A. grupal.

2. Metodología de aprendizaje 5R (aplicada)

4. R1 – REGISTRAR (Copiar y compilar): se copió el código en src/main.cpp, se compiló con «g++ -std=c++11 src/main.cpp -o src/piedrapapelotijera.exe» y se ejecutó con «.\src\piedrapapelotijera.exe». Se verificó la generación de src/historial_partidas.txt.
5. R2 – REDUCIR (Núcleo): líneas clave «ofstream archivo("src/historial_partidas.txt", ios::app);» (escritura) e «ifstream archivo("src/historial_partidas.txt");» (lectura).
6. R3 – RESPONDER: cuestionario técnico respondido en la Sección 4 de este informe.
7. R4 – REFLEXIONAR: la misma técnica permite guardar configuraciones o logs de hardware en .txt/.csv (ver Sección 5).
8. R5 – REVISAR: se verificó la estructura de carpetas src/, docs/, docs/manuales/ y capturas/

3. Desarrollo: diseño y persistencia

3.1 Estructura Partida

struct Partida { int idPartida; string jugador; string eleccionJugador; string eleccionIA; string resultado; }; Agrupa variables heterogéneas en una sola entidad que se pasa por referencia a guardarEnDisco(const Partida&).

3.2 Escritura (ofstream + ios::app)

Cada partida se agrega al final del archivo en formato CSV: «idPartida,jugador,eleccionJugador,eleccionIA,resultado». El modo ios::app evita el truncado. Se verifica is_open() y se informa por cerr si falla. Luego se cierra con close().

3.3 Lectura (ifstream)

Se abre con ifstream y se lee secuencialmente con getline(archivo, campo, ',') para los cuatro primeros campos y getline(archivo, resultado) para el último. Se cuenta el total y se muestra. Si el archivo no existe se informa que aún no hay partidas; si está vacío se avisa explícitamente.

3.4 Búfer de teclado

Después de «cin >> opcion» queda un salto de línea en el búfer. Por eso jugarPartida() invoca cin.ignore() antes de getline(cin, p.jugador); de lo contrario el nombre se leería vacío.

3.5 Comandos utilizados

Compilación: g++ -std=c++11 src/main.cpp -o src/piedrapapelotijera.exe. Ejecución: .\src\piedrapapelotijera.exe.

4. Cuestionario Técnico de Control

Pregunta 1 — RAM vs. disco: ¿por qué al cerrar la consola y reabrir el ejecutable la Opción 2 sigue mostrando el historial?

Respuesta: Porque el historial no vive en la RAM sino en el archivo src/historial_partidas.txt en disco (memoria no volátil). Las variables se pierden al cerrar el programa, pero el archivo permanece. La Opción 2 lo reabre con ifstream y lo vuelve a mostrar, por eso persiste entre ejecuciones.

Pregunta 2 — ¿Qué pasaría si en guardarEnDisco se usara ofstream archivo("historial_partidas.txt"); sin ios::app?

Respuesta: Sin ios::app el archivo se abre en modo out por defecto, que trunca (borra) el contenido existente. Cada partida nueva sobrescribiría a las anteriores y solo quedaría guardada la última. Con ios::app los datos se agregan al final y se conservan todas las partidas.

Pregunta 3 — ¿Por qué es necesario cin.ignore() antes de getline(cin, p.jugador)?

Respuesta: Porque «cin >> opcion» (y luego «cin >> opcJugador») deja el carácter Enter (\n) pendiente en el búfer de entrada. Si se llama a getline() directamente, lee ese resto vacío y 'saltea' la pregunta del nombre. cin.ignore() limpia ese carácter para que getline() lea el nombre real, incluso con espacios.

Pregunta 4 — ¿Qué ventajas ofrece struct Partida frente a pasar 5 variables sueltas?

Respuesta: Orden y cohesión: una sola entidad representa la partida en vez de cinco parámetros sueltos; las funciones reciben un solo argumento (const Partida&). Legibilidad y mantenimiento: agregar un campo (por ejemplo fecha) no cambia la firma de las funciones. Mapeo directo con el CSV (una línea = una Partida) y menor riesgo de errores de orden de parámetros.

Pregunta 5 — Aplicación al P.I.A.: ¿qué datos guardarían del prototipo en el archivo de texto?

Respuesta 5: Una posible aplicación sería adaptar el módulo a mi prototipo, un robot seguidor de línea activado por app. Se podría reutilizar la estructura como `EventoRobot` y guardar en un `.txt`, una línea por evento, datos como encendido/apagado desde la app, pérdida/recuperación de línea de los sensores IR, batería y velocidad. Esto permitiría llevar un log persistente para depurar fallas, con la misma lógica de `ofstream` en `ios::app` e `ifstream` del juego.

5. Reflexión (R4) y glosario

La técnica struct + fstream es reutilizable para cualquier log persistente: configuraciones, lecturas de sensores, usuarios o stock. El archivo .txt/.csv actúa como base de datos mínima, legible y portable.

- Memoria RAM (volátil): espacio temporal; al terminar el programa los datos en variables se pierden.
- Persistencia en disco (no volátil): escritura en almacenamiento secundario (.txt/.dat/.csv) para conservar datos.
- <fstream>: librería estándar de flujos de archivos.
- ofstream: flujo de salida (programa → archivo).
- ifstream: flujo de entrada (archivo → programa).
- ios::app (append): agrega al final sin borrar lo existente.
- cin.ignore(): limpia el Enter pendiente antes de getline().

6. Estructura entregada y verificación

Estructura local entregada: .gitattributes, .gitignore, LICENSE, README.md, docs/ (informe PDF + CHANGELOG.md), docs/manuales/ (manuales de usuario y programador en MD y PDF), src/main.cpp, src/historial_partidas.txt y carpeta capturas (con imagenes del juego y del historial). Compilación verificada con g++ -std=c++11 y lectura del historial verificada con la Opción 2 (3 registros de prueba).

- src/main.cpp — programa principal compilable.
- src/historial_partidas.txt — persistencia generada (CSV).
- docs/ — informe + CHANGELOG.md.
- docs/manuales/ — manual_usuario y manual_programador (MD + PDF).
- capturas/ — imagenes del juego y del historial.